Coding & Solution

Smart Lender | Team Lead: Venkat Rajarapu | Contributor: Kollipara TejaHansika

Implementation Summary

The Smart Lender solution was implemented across five epics, moving from raw data to a working web application.

Epic 1: Data Collection and Architecture Design

- Loan applicant dataset identified and stored under data/loan_data.csv.
- Application architecture defined: HTML frontend, Flask backend, trained model, prediction.

Epic 2: Visualizing and Analysing the Data

- Dataset imported with pandas; basic structure and types inspected.
- Univariate analysis: distribution plots for income, loan amount, credit history.
- Bivariate analysis: relationship between Education/Property Area and Loan_Status.
- Multivariate analysis: correlation heatmap across numeric features.

Epic 3: Data Pre-processing

- Missing values handled - mode for categorical columns, median for numeric columns.
- Class imbalance addressed using SMOTE oversampling on the training set.
- Features scaled using StandardScaler for distance-based models (KNN).
- Dataset split 80/20 into training and test sets with stratification.

Epic 4: Model Building

- Four classifiers trained and compared: Decision Tree, Random Forest, KNN, XGBoost.
- Each model evaluated on accuracy, precision, recall, and F1-score.
- Best performing model serialized to model.pkl using pickle.

Epic 5: Application Building

- Flask app (app.py) built with two routes: / for the form and /predict for inference.
- HTML templates (index.html, result.html) built for input and output display.
- Application tested locally via python app.py and verified end-to-end.